Tema Integrador

Sistema Academia Popular - Projeto: NÚCLEOS CARIOCAS

Fabiano H. Carminatti Jr.
Ciência da computação EAD - 4º Período
Profª Orientadora: Denise Moraes

Identificação do projeto

Nome(s) do(s) aluno(s): Fabiano Henrique Carminatti Junior.
Curso: Ciência da Computação (EAD).
Tema: Tema Integrador - Projeto Sistemas e Banco de Dados.
Local de realização do projeto: Associação de moradores de Oswaldo Cruz.
Público – alvo: Alunos do projeto **NÚCLEOS CARIOCAS**.

Introdução

Sobre o Projeto

O **NÚCLEOS CARIOCAS** é um projeto da Secretaria Municipal de Assistência Social do Rio de Janeiro, que visa promover a inserção social em áreas de vulnerabilidade.
Atualmente, são mais de 75 núcleos em toda a cidade, cujas atividades beneficiam crianças, adolescentes, adultos e idosos.
Cada núcleo oferece atividades de cultura, esporte e lazer, com o objetivo de aumentar a autoestima e a autonomia dos participantes.
As atividades são desenvolvidas de acordo com as necessidades locais e são realizadas em parceria com os Centros de Referência em Assistência Social (CRAS), que encaminham os usuários às atividades.
Para esse projeto, estamos atuando na academia popular, realizada na Associação de Moradores de Oswaldo Cruz.
O responsável pelo projeto em Oswaldo Cruz é o Segio Corrêa, professor de Educação Física e quem ministra as aulas.

Coleta de dados

Realizei novas entrevistas com o professor Sérgio, que validou as informações presentes nesse documento e me deu suporte com dúvidas através do Whatsapp.
Relembrando que, durante as entrevistas, foram observados os seguintes desafios:

- **Falta de centralização de informações**: Não existe nenhum sistema atualmente para armazenar os dados dos alunos (informações pessoais, necessidades de cada aluno, presenças, etc);
- **Divulgação limitada**: o projeto é divulgado para as pessoas por boca a boca, dificultando o acesso a novos alunos;
- **Dificuldade com novas tecnologias**: o público que participa do projeto é em maioria idoso, que não são muito familiarizados com tecnologias complexas
- **Comunicação dispersa**: atualmente, toda a comunicação é feita via whatsapp, fazendo com que informações importantes se percam no histórico.

ODS 17 — Parcerias e Meios de Implementação

Essa ODS fala sobre:

- cooperação;
- parcerias;
- integração entre comunidade, instituições e sociedade;

- compartilhamento de conhecimento e tecnologia para melhorar a qualidade de vida das pessoas.

Essa ODS se relaciona com o nosso trabalho, pois ele foi pensado para apoiar um projeto social comunitário voltado ao público idoso, promovendo a colaboração e melhoria na gestão das atividades realizadas.

Além disso, o sistema proposto contribui para facilitar a comunicação entre professores e alunos, centralizar informações importantes e ampliar a acessibilidade digital para os participantes do projeto. Dessa forma, a iniciativa demonstra como a tecnologia pode ser utilizada para fortalecer ações sociais e apoiar a inclusão da população idosa em atividades comunitárias.

Objetivo

Esse trabalho é a continuação do tema integrador anterior. Através de ferramentas, como o MariaDB e o Flask, irei, com a orientação da professora Denise, criar uma aplicação web para registro e controle de dados dos alunos e professores do projeto Núcleos Cariocas. Começamos o projeto com a criação de um banco de dados, que se conectará ao html através do python. Utilizando também a interface de rotas do Flask, a aplicação permite cadastro, alteração, exclusão e busca no banco da dados criados.
O foco é que todo o sistema funcione, com o mínimo de erros possível. O design ainda está um pouco “cru”, mas espero melhora-lo em próximas versões.

Atividades realizadas

Levantamento dos requisitos Funcionais e Não Funcionais.

Requisitos Funcionais

RF01: Cadastro de Alunos
O sistema deve permitir ao professor o cadastro de novos alunos com informações básicas (nome, idade, telefone, endereço, peso, altura, sexo).

RF02: Cadastro de Aulas
O sistema deve permitir ao professor cadastrar aulas com data, horário e descrição das atividades.

RF03: Exibição de Agenda
O sistema deve permitir que os alunos visualizem os dias e horários das aulas cadastradas.

RF04: Registro de Presença
O sistema deve permitir que o aluno indique presença em uma aula através de um botão (“Compareci”).

RF05: Validação de Presença
O professor deve poder aprovar, rejeitar ou corrigir registros de presença realizados pelos alunos.

RF06: Histórico de Participação
O sistema deve armazenar o histórico de presenças dos alunos para consulta posterior.

RF07: Acompanhamento de Dados Pessoais
O aluno deve poder visualizar seus dados pessoais e seu histórico de frequência. Quando houver atualização de peso ou medidas, o sistema poderá exibir a evolução de forma gráfica.

RF08: Emissão de Avisos
O professor deve poder emitir avisos ou comunicados que são exibidos para todos os alunos cadastrados da turma.

RF09: Notificações
O sistema deve exibir notificações internas sobre novas aulas e avisos publicados.

RF10: Login Simplificado
O sistema deve permitir o acesso por meio de autenticação simples (telefone e senha ou PIN).

RF11: Perfis de Usuário
O sistema deve diferenciar perfis de aluno e professor, exibindo funcionalidades específicas para cada tipo.

RF12: Cadastro Manual de Presença
O sistema deve permitir que o professor registre manualmente a presença de alunos caso necessário.

RF13: Gerenciamento de Turmas
O sistema deve permitir a criação e gerenciamento de turmas pelo professor.

RF14: Visualização de Relatórios
O professor deve poder visualizar relatórios básicos com número de alunos ativos e presenças mensais.

RF15: Atualização e Exclusão de Cadastros
O sistema deve permitir ao professor editar ou excluir registros de alunos, aulas, turmas e avisos quando necessário.

RF16: Consulta de Alunos
O sistema deve permitir ao professor consultar a lista de alunos cadastrados, com possibilidade de visualizar dados individuais.

Requisitos não funcionais

RNF01: Acessibilidade
O sistema deve ter botões grandes, letras legíveis e ícones intuitivos, visando o público idoso.

RNF02: Usabilidade
A interface deve ser simples e com poucos passos.

RNF03: Portabilidade
O sistema deve funcionar em dispositivos Android e iOS, priorizando smartphones.

RNF04: Desempenho
O aplicativo deve carregar a tela principal rapidamente em conexão 3G.

RNF05: Armazenamento Offline
O sistema deve prever armazenamento local temporário para consulta de informações básicas em caso de instabilidade na internet.

RNF06: Segurança
Os dados dos usuários devem ser armazenados de forma segura, respeitando os princípios da LGPD.

RNF07: Escalabilidade
O sistema deve permitir a adição de novos grupos e turmas sem reconfiguração manual.

RNF08: Compatibilidade
O sistema deve ser compatível com versões do Android 8.0 ou superior e iOS 13 ou superior.

RNF09: Responsividade
A interface deve se ajustar automaticamente a diferentes tamanhos de tela.

RNF10: Manutenibilidade
O código deve ser modular para facilitar futuras atualizações e correções.

RNF11: Notificações Simples
As notificações devem ser claras e diretas, sem exigir interação complexa do usuário.

RNF12: Idioma
Todo o conteúdo deve estar em português e utilizar linguagem clara e objetiva.

Levantamento das Regras de Negócio

RN01: Apenas professores podem cadastrar, editar ou excluir alunos, aulas, turmas e avisos.

RN02: Alunos podem visualizar a agenda, consultar seus próprios dados e registrar presença nas aulas.

RN03: Cada aluno deve estar vinculado a pelo menos uma turma.

RN04: Cada aula deve estar vinculada a uma turma.

RN05: A presença registrada pelo aluno deve ficar pendente até validação do professor.

RN06: O professor pode aprovar, rejeitar ou corrigir uma presença registrada.

RN07: Um aluno não pode registrar presença duplicada na mesma aula.

RN08: Avisos devem ser associados a uma turma ou publicados para todos os alunos.

RN09: O telefone deve ser único para cada usuário, pois será usado no login simplificado.

RN10: Relatórios devem considerar apenas alunos ativos.

RN11: Os dados pessoais dos alunos devem ser acessíveis apenas pelo próprio aluno e pelo professor responsável.

RN12: O sistema deve manter histórico das presenças mesmo após alterações cadastrais.

RN13: Todos os professores devem possuir obrigatoriamente CREF.

Diagrama de Casos de Uso

Ferramenta usada: lucid.app

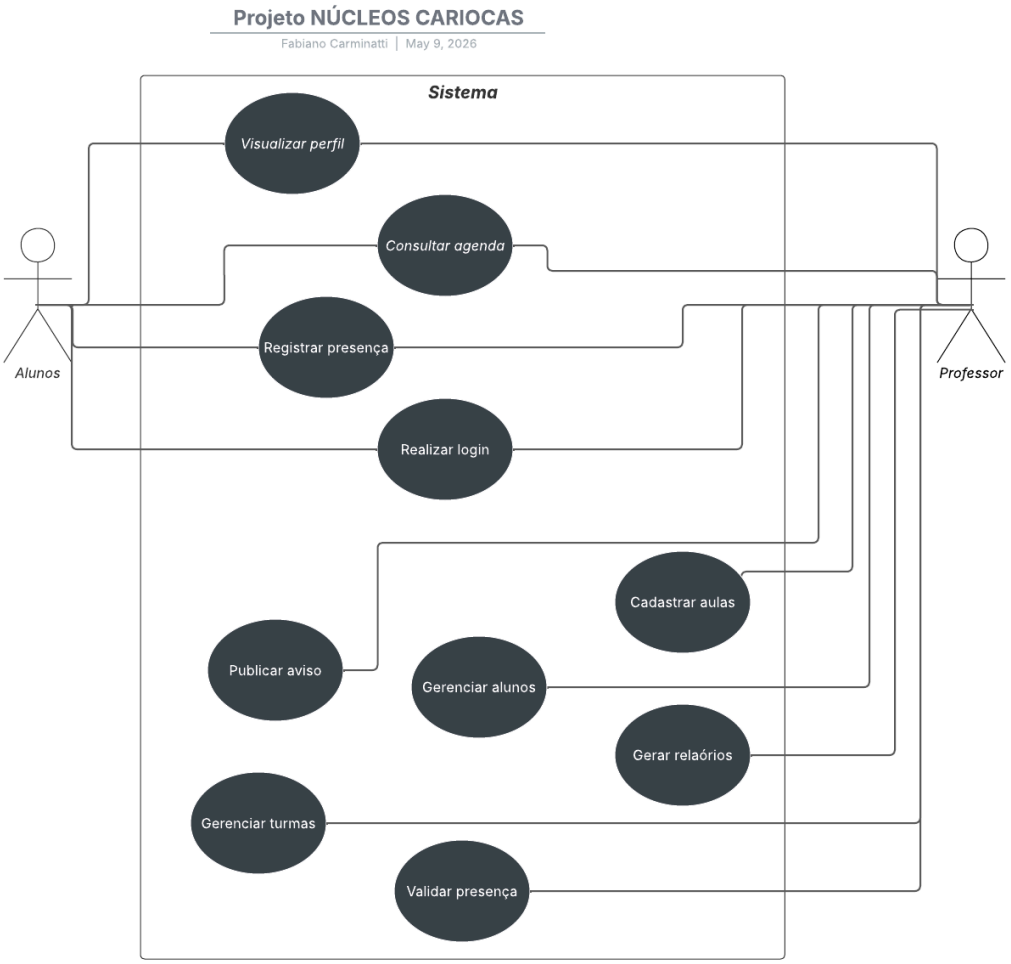
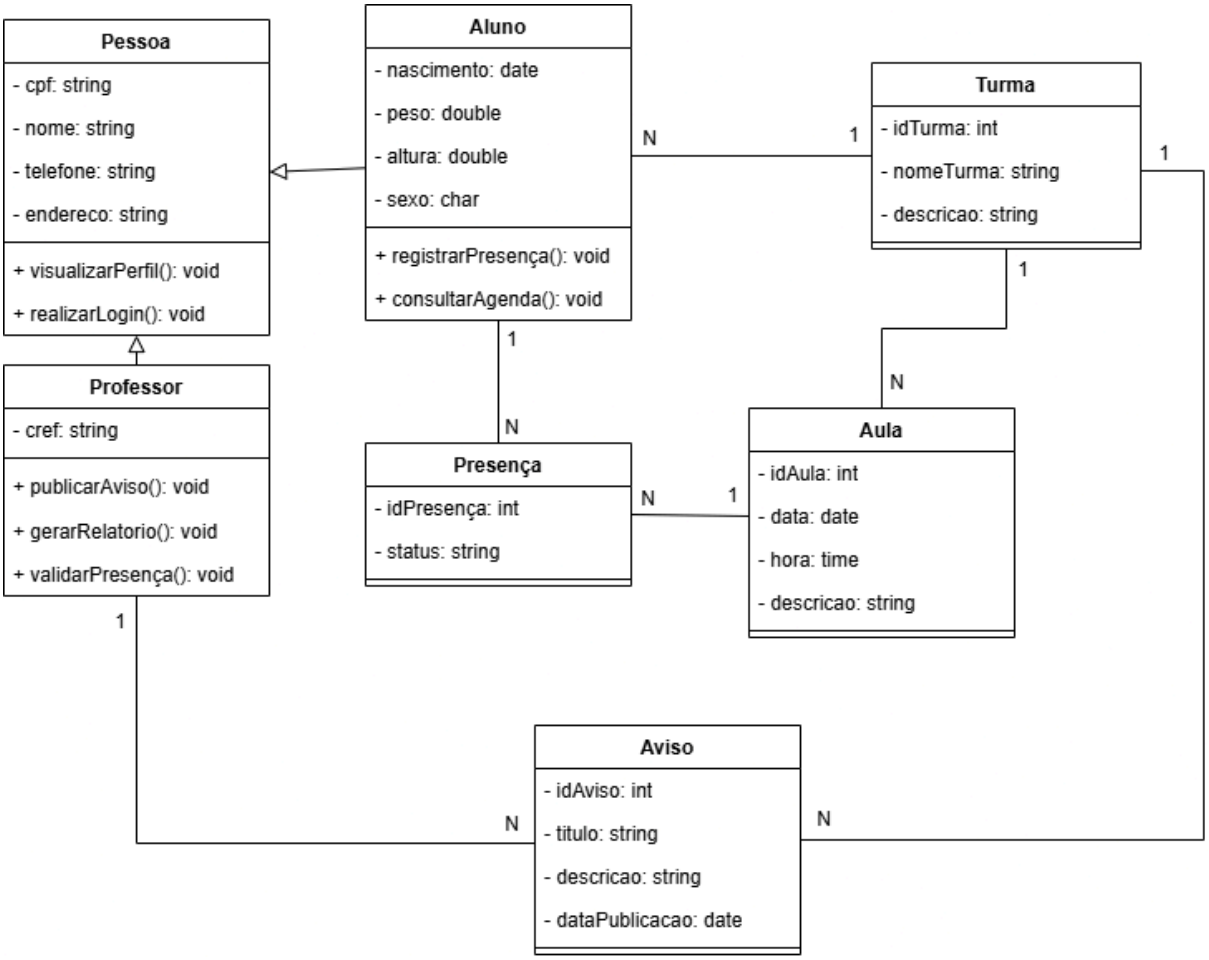


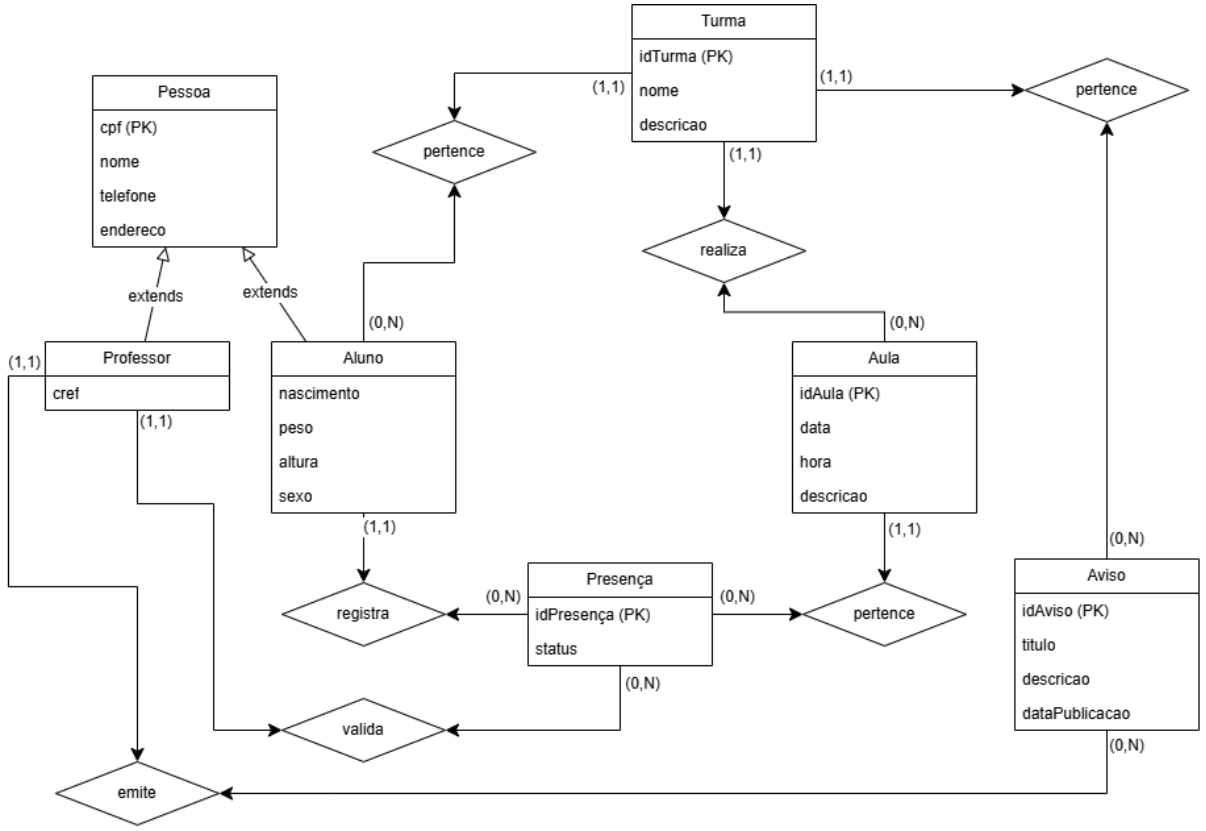
Diagrama de Classes

Ferramenta usada: draw.io



Modelo Entidade Relacionamento

Ferramenta usada: draw.io



CRUD

Criação do banco e suas tabelas

Para o sistema, pensei nas seguintes tabelas:

- pessoa (cpf, nome, telefone, endereco, senha_hash)
- aluno (cpf, nascimento, peso, altura, sexo, idTurma)
- professor (cpf, cref)
- turma (idTurma, nome, descricao)
- aula (idAula, data, hora, descricao, idTurma)
- presenca (idPresenca, status, cpfAluno, idAula, cpfProfessorValidador)
- aviso (idAviso, titulo, descricao, dataPublicacao, idTurma, cpfProfessor)

Ferramentas usadas: MariaDB, HeidiSQL

Começaremos criando um banco de dados, chamado de nucleos_cariocas.

```
CREATE DATABASE nucleos_cariocas;

USE nucleos_cariocas;
```

Criando as tabelas.

```
CREATE TABLE pessoa (
        cpf VARCHAR(11) PRIMARY KEY,
        nome VARCHAR(100) NOT NULL,
        telefone VARCHAR(15) UNIQUE NOT NULL,
        endereco VARCHAR(100),
        senha_hash VARCHAR(255) NOT NULL
);

CREATE TABLE turma (
        idTurma INT AUTO_INCREMENT PRIMARY KEY,
        nome VARCHAR(100) NOT NULL,
        descricao VARCHAR(100)
);

CREATE TABLE aluno (
        cpf VARCHAR(11) PRIMARY KEY,
        nascimento DATE,
        peso DOUBLE,
        altura DOUBLE,
        sexo CHAR(1),
        idTurma INT,
        FOREIGN KEY (cpf) REFERENCES pessoa(cpf),
        FOREIGN KEY (idTurma) REFERENCES turma(idTurma)
);
```

```
CREATE TABLE professor (
        cpf VARCHAR(11) PRIMARY KEY,
        cref VARCHAR(11) UNIQUE NOT NULL,
        FOREIGN KEY (cpf) REFERENCES pessoa(cpf)
);


CREATE TABLE aula (
        idAula INT AUTO_INCREMENT PRIMARY KEY,
        data DATE,
        hora TIME,
        descricao VARCHAR(100) NOT NULL,
        idTurma INT,
        FOREIGN KEY (idTurma) REFERENCES turma(idTurma)
);

CREATE TABLE aviso (
        idAviso INT AUTO_INCREMENT PRIMARY KEY,
        titulo VARCHAR(30) NOT NULL,
        descricao VARCHAR(100) NOT NULL,
        dataPublicacao DATE,
        idTurma INT,
        cpfProfessor VARCHAR(11),
        FOREIGN KEY (idTurma) REFERENCES turma(idTurma),
        FOREIGN KEY (cpfProfessor) REFERENCES professor(cpf)
);

CREATE TABLE presenca (
        idPresenca INT AUTO_INCREMENT PRIMARY KEY,
        status VARCHAR(20) DEFAULT 'PENDENTE',
        cpfAluno VARCHAR(11) NOT NULL,
        idAula INT NOT NULL,
        cpfProfessorValidador VARCHAR(11),
        FOREIGN KEY (cpfAluno) REFERENCES aluno(cpf),
        FOREIGN KEY (idAula) REFERENCES aula(idAula),
        FOREIGN KEY (cpfProfessorValidador) REFERENCES professor(cpf)
);
```

Para o modelo CRUD, usarei como base a tabela de “turma”

CREATE

Em **turma_repository.py**:

```
def salvar_turma(nome, descricao):
    conexao = conectar()
    cursor = conexao.cursor()

    sql = "INSERT INTO turma (nome, descricao) VALUES (%s, %s)"
    valores = (nome, descricao)
    cursor.execute(sql, valores)
    conexao.commit()

    id_nova_turma = cursor.lastrowid

    print("Turma cadastrada com sucesso!")

    cursor.close()
    conexao.close()

    return id_nova_turma
```

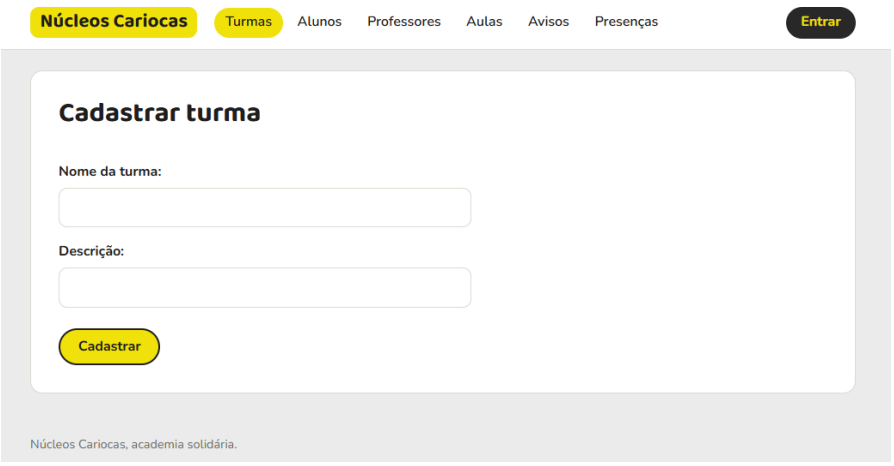
Em **main.py**:

```
@app.route("/turmas/cadastrar", methods=["GET", "POST"])
def cadastrar_turma():
    if request.method == "POST":
        nome = request.form.get("nome")
        descricao = request.form.get("descricao")

        id_turma = salvar_turma(nome, descricao)

        return redirect(url_for("visualizar_turma", id_turma=id_turma))

    return render_template("cadastrar_turma.html")
```



READ

Em **turma_repository.py**:

```
def listar_turmas():
    conexao = conectar()
    cursor = conexao.cursor()

    sql = "SELECT * FROM turma"
    cursor.execute(sql)

    turmas = cursor.fetchall()

    cursor.close()
    conexao.close()

    return turmas
```

Em **main.py**:

```
@app.route("/turma/lista", methods=["GET"])
def lista_de_turmas():
    turmas = listar_turmas()

    return render_template("listar_turma.html", turmas=turmas)
```



UPDATE

Em **turma_repository.py**:

```
def atualizar_turma(id_turma, nome, descricao):
    conexao = conectar()
    cursor = conexao.cursor()

    sql = "UPDATE turma SET nome = %s, descricao = %s WHERE idTurma = %s"
    valores = (nome, descricao, id_turma)
    cursor.execute(sql, valores)
    conexao.commit()

    print("Turma atualizada com sucesso!")

    cursor.close()
    conexao.close()
```

Em **main.py**:

```
@app.route("/turmas/editar/<int:id_turma>", methods=["GET", "POST"])
def editar_turma(id_turma):
    turma = buscar_turma(id_turma)

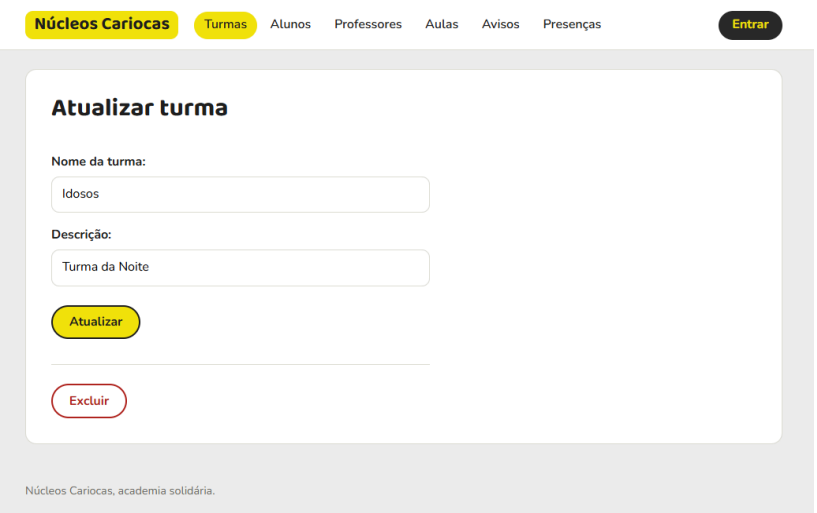
    if turma is None:
        return "Turma não encontrada!", 404

    if request.method == "POST":
            nome = request.form.get("nome")
            descricao = request.form.get("descricao")

            atualizar_turma(turma.idTurma, nome, descricao)

            return redirect(url_for("visualizar_turma", id_turma=turma.idTurma))

    return render_template("atualizar_turma.html", turma=turma)
```



DELETE

Em **turma_repository.py**:

```
def deletar_turma(id_turma):
    conexao = conectar()
    cursor = conexao.cursor()

    sql = "DELETE from turma WHERE idTurma = %s"
    valor = (id_turma,)
    cursor.execute(sql, valor)
    conexao.commit()

    print("Turma excluída com sucesso!")

    linhas_afetadas = cursor.rowcount

    cursor.close()
    conexao.close()

    return linhas_afetadas
```

Em **main.py**:

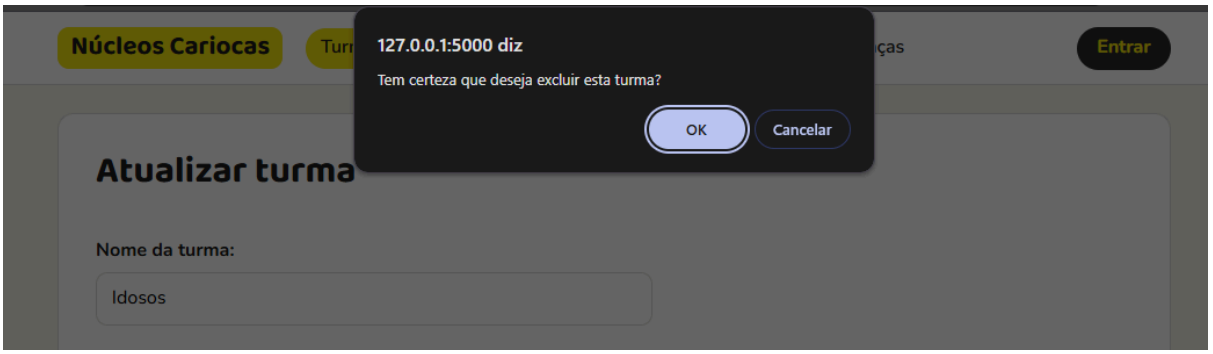
```
@app.route("/turmas/excluir/<int:id_turma>", methods=["POST"])
def excluir_turma(id_turma):
    turma = buscar_turma(id_turma)

    if turma is None:
        return "Turma não encontrada!", 404

    if request.method == "POST":
        linhas_afetadas = deletar_turma(id_turma)

        if (linhas_afetadas == 0):
            print("regristro não encontrado e/ou já excluído")
        else:
            print("Exclusão realizada com sucesso!")

    return redirect(url_for("cadastrar_turma"))
```



Conclusão

Ao longo deste projeto, consegui compreender melhor as etapas envolvidas no desenvolvimento de um sistema, desde o levantamento de requisitos até a modelagem e implementação do banco de dados. Achei bem legal a ideia de CRUD.

Já nesse ciclo de aprendizagem, pude ampliar a aplicação para um modelo mais funcional do que simplesmente um banco de dados. Agora, o sistema tem uma “cara”, uma interface onde é possível um usuário interagir com a aplicação.

Pude melhorar os conceitos de python que eu aprendi durante o período, aprendi sobre como realizar a integração com um banco de dados, e os cuidados que devemos ter para manter a integridade do banco e a solidez da aplicação.

Aprendi a usar o ambiente virtual (.venk) onde foii possível gerencias dependências específicas sem conflitar com futuros projetos que eu for desenvolver.

A utilização de Flask também foi um ponto forte, pois facilitou muito a interface e a comunicação com os códigos python. Estou muito feliz com meu primeiro projeto e pretendo trazer atualizações futuras que ficaram pendentes, como o login na plataforma e um tratamento adequado para as senhas (que deveriam usar hash). Além de uma formatação mais “bonita” de campos, como telefone e cpf.

Funcionalidades implementadas na versão 1.0

- Criação do DB e suas tabelas;
- Criação das Classes e seus repositórios;
- Integração do banco de dados (MariaDB) e html usando Python e Flask;
- Criação de uma interface básica para a aplicação usando Flask;
- Todas as 6 classes da aplicação com seus devidos CRUDs implementados e textos.

Funcionalidades a serem implementadas em versões futuras

- Login e controle de usuários;
- Tratamento de senhas usando hash;
- Formatação dos campos de cpf, cred, telefone;
- Adicionar seletores em campos que precisam existir no DB, para auxiliar o preenchimento;
- Mlhorias de acessibilidade e responsividade;
- Relatórios de frequência e participação;

Ferramentas de inteligência artificial utilizadas

Durante o desenvolvimento do sistema Núcleos Cariocas, utilizei ferramentas de inteligência artificial como apoio ao processo de aprendizagem e implementação da aplicação.

O **ChatGPT (OpenAI)** foi utilizado como ferramenta de tutoria, esclarecimento de conceitos, revisão de código e auxílio na identificação e correção de erros, especialmente em Python, Flask, SQL e integração com o banco de dados.

O **Claude (Anthropic)** foi utilizado como apoio na elaboração da interface gráfica, incluindo a organização visual das páginas e a estilização com HTML, CSS e templates Jinja2.

O desenvolvimento do projeto envolveu decisões, implementação, testes e revisões que foram feitas por mim, que utilizei as ferramentas para compreender os conceitos e aprimorar a aplicação, além de aulas disponibilizadas gratuitamente no youtube.

As contribuições das ferramentas foram incorporadas ao projeto com acompanhamento e avaliação minha.

Link para o github: https://github.com/taygolino/Tema_Integrador_Nucleos_Carioca

Referências

Ferramentas utilizadas

- MariaDB
- SQL
- HeidiSQL
- Draw.io
- Lucid.app
- GitHub
- Python 3.15.2
- Visual Studio Code
- Flask
- ChatGPT
- ClaudeIA

Fontes de pesquisa

IBM, **Diagramas de Classes**. Disponível em: https://www.ibm.com/docs/pt-br/rsas/7.5.0?topic=structure-class-diagrams. Acesso em: 10 maio 2026.

LUCIDCHART, **O que é um diagrama de classe UML?**. Disponível em: https://www.lucidchart.com/pages/pt/o-que-e-diagrama-de-classe-uml. Acesso em: 10 maio 2026.

LUCIDCHART, **Diagrama de caso de uso UML: O que é, como fazer e exemplos**. Disponível em: https://www.lucidchart.com/pages/pt/diagrama-de-caso-de-uso-uml. Acesso em: 10 maio 2026.

DEVMEDIA, **Orientações básicas na elaboração de um diagrama de classes**. Disponível em: https://www.devmedia.com.br/orientacoes-basicas-na-elaboracao-de-um-diagrama-de-classes/37224. Acesso em: 10 maio 2026.

DEVMEDIA, **O que é UML e Diagramas de Caso de Uso: Introdução Prática à UML**. Disponível em: https://www.devmedia.com.br/o-que-e-uml-e-diagramas-de-caso-de-uso-introducao-pratica-a-uml/23408. Acesso em: 10 maio 2026.

OPENAI. **Chat GPT**. Assistente virtual baseado em inteligência artificial. Disponível em: https://chat.openai.com/. Acesso em: 08 novembro 2026.

Anthropic. **Claude IA**. Assistente virtual baseado em inteligência artificial. Disponível em: https://claude.ai/new. Acesso em: 08 novembro 2026.

Hashtag Programação. Youtube. **Aprenda Flask em 10min (Criar Sites em Python)**. Disponível em: https://www.youtube.com/watch?v=WnaRrfqq5uw. Acesso em: 08 novembro 2026

Evidências









Portfólio do Projeto Integrador





